Apache Airflow - projekt 1

Apache Airflow to platforma *open source* do opracowywania, harmonogramowania i monitorowania wsadowych przepływów zadań (*workflows*). Funkcjonalność, którą ona oferuje, często określa się mianem orkiestracji.

Niektórzy uważają, że jest to jedno z najlepszych narzędzi w swojej klasie.

Apache Airflow jest oparty na Pythonie. Prawie w każdym elemencie systemu wykorzystywany jest właśnie ten język programowania. Począwszy od sposobu definiowania przepływów przez użytkownika, a skończywszy na implementacji wtyczek. Programistów często zaskakuje zakres możliwości *Apache Airflow*, który pozwala na tworzenie przepływów wykorzystujących praktycznie dowolną technologię.

Istotnym elementem *Apache Airflow* jest interfejs sieciowy. Pozwala on nie tylko podglądać zdefiniowane przepływy oraz je uruchamiać, ale także w pełnym zakresie zarządzać ich stanem.

Apache Airflow można wdrożyć na wiele sposobów, od pojedynczego procesu na laptopie po konfigurację rozproszoną, która obsługuje potężne przepływy w dużej ich liczbie. My wykorzystamy bardzo prostą konfigurację uruchomioną na maszynie driver (master) naszego klastra - w chmurze albo w ramach kontenera `master` naszego środowiska *Dockerowego*.

Celem tego warsztatu będzie przygotowanie przepływu, który będzie zarządzał uruchamianiem (orkiestrował) elementów składowych projektu 1, a następnie przygotowanie pełnego zestawu składowych do rejestracji projektu w ramach kursu.

Elementy wyjściowe

W założeniu wszystkie elementy składowe dla projektu 1 są już gotowe. W przypadku niniejszego warsztatu będzie to wariant: *Hadoop Streaming*, *Zestaw danych nr 0*.

W założeniu mamy już gotowe:

- Program *MapReduce*:
  - skrypt mappera: `mapper.py`
  - skrypt reduktora: `reducer.py`
  - skrypt uruchamiający program *MapReduce*: `run_mr.sh`

- Program *Hive*:
  - skrypt z poleceniami HQL: `hive.hql`
  - skrypt uruchamiający program *Hive*: `run_hive.sh`
- Dane wejściowe:
  - dostępne w ich oryginalnym miejscu wskazanym w ramach kursu (w przypadku *Google Cloud Platform* – w Twoim zasobniku).

Ponadto zakładamy, że dostępna jest źródłowa definicja przepływu *Apache Airflow*, plik `projekt1.py`.

Zwróć uwagę, że każdy projekt jest inny, oparty o inny zestaw danych, zawierający składowe specyficzne dla określonego wariantu. Dlatego nie traktuj tego warsztatu "dosłownie". Każdorazowo uwzględnij elementy specyficzne dla Twojego projektu i odpowiednio dostosuj podejmowane działania. W razie potrzeby szukaj wsparcia u prowadzącego. Nie zmieniaj jednak elementów wspólnych.

Wszystkie elementy składowe projektu (wymienione wyżej) powinny być wcześniej przetestowane, a ich poprawne działanie powinno być potwierdzone. Polecenia uruchamiające poszczególne programy powinny być opracowane, znane i działające.

W razie potrzeby, zanim zaczniesz ten warsztat, wprowadź odpowiednie poprawki oraz przetestuj jeszcze raz wszystkie składowe projektu. Alternatywnie możesz próbować to robić na bieżąco... choć może to być trudniejsze.

Przygotowanie środowiska

Sposób przygotowania środowiska różni się w zależności od tego, z której platformy korzystasz - wybierz właściwą kartę poniżej.

Managed Apache Spark (Google Cloud Platform)

Apache Airflow nie należy do komponentów dostępnych w ramach klastra *Managed Service for Apache Spark* (dawniej *Dataproc*), o czym możemy się przekonać na stronie: `https://docs.cloud.google.com/managed-spark/docs/concepts/versioning/image-release-3.0`.

Oznacza to, że będziemy go musieli zainstalować niezależnie na uruchomionym uprzednio klastrze.

Tak na marginesie, w ramach usług dostarczanych przez platformę GCP istnieje narzędzie *Cloud Composer*. Jest to usługa pozwalająca na orkiestrację przetwarzania (danych). Opiera

się ona całkowicie na *Apache Airflow*. Nie będziemy jednak z niej korzystali w ramach tego warsztatu.

1. Otwórz konsolę GCP `https://console.cloud.google.com/`
2. Następnie uruchom terminal *Cloud Shell*.
3. Korzystając z sesji *Cloud Shell* utwórz klaster *Managed Service for Apache Spark*.

> **UWAGA**
>
> Jeśli poniższe zmienne zostały przez Ciebie ustawione, np. za pomocą wpisów w pliku `~/.bashrc` , możesz pominąć ich ponowne ustawianie poniżej.

```
export REGION=tu-wprowadz-nazwe-wybranego-regionu
export CLUSTER_NAME=pbd-cluster
export PROJECT_ID=$(gcloud config get-value project)
```

```
gcloud dataproc clusters create ${CLUSTER_NAME} --enable-component-gateway
\
--region ${REGION} --subnet default --public-ip-address \
--master-machine-type n2-standard-4 --master-boot-disk-size 50 \
--num-workers 2 --worker-machine-type n2-standard-2 --worker-boot-disk-
size 50 \
--image-version 3.0-debian13 --project ${PROJECT_ID} --max-age=3h
```

4. Na zakładce *Nodes* (dawniej VM *Instances*) szczegółów klastra uruchom terminal SSH do maszyny driver (master) naszego klastra.



Docker (środowisko lokalne)

5. Uruchom maszynę, w której znajduje się przygotowane środowisko.
6. Otwórz terminal linii komend.
7. Sprawdź i ewentualnie uruchom nasze środowisko.
   a. W terminalu przejdź do katalogu `BigData25` zawierającego konfigurację naszego środowiska *Dockerowego*.

```
cd ~/BigData25
```

```
admin@debian:~$ cd ~/BigData25
admin@debian:~/BigData25$ ls docker-compose.yml
docker-compose.yml
```

b. Uruchom polecenie sprawdzające, jakie kontenery w ramach naszego środowiska już istnieją i czy zostały już uruchomione.

```
docker compose ps -a --format "{{.ID}} {{.Image}} {{.Names}} {{.Status}}"
```

c. Jeśli jakieś kontenery istnieją, to przed dalszymi działaniami (o ile nie kontynuujesz wcześniejszej pracy) usuń nasze środowisko za pomocą poniższego polecenia.

```
docker compose down -v
```

d. Zanim uruchomimy nasze środowisko, dodamy dodatkowy port udostępniany przez kontener `master`. W tym celu otwórz *Visual Studio Code*. Otwórz plik `~/BigData25/docker-compose.yml` zawierający konfigurację naszego środowiska. Znajdź sekcję `services/master/ports`, a następnie dodaj do listy portów poniższą linię. Koniecznie zapisz wprowadzone zmiany.

```
- "8080:8080"  # Apache Airflow
```

```
ports:
  - "4040:4040" # Spark Driver UI
  - "8020:8020" # HDFS NameNode RPC port
  - "8080:8080" # Apache Airflow
  - "8088:8088" # YARN ResourceManager UI
  - "9870:9870" # HDFS NameNode Web UI
  - "9999:9999" # Tez UI (static files)
```

e. Uruchom środowisko poniższym poleceniem.

```
docker compose up -d
```

f. Upewnij się, czy wszystkie kontenery zostały poprawnie uruchomione.

```
docker compose ps -a --format "{{.ID}} {{.Image}} {{.Names}}
{{.Status}}"
```

```
admin@debian:~/BigData25$ docker compose ps -a --format "{{.ID}} {{.Image}} {{.Names}} {{.Status}}"
e6382e644693 bigdata25-master:latest bigdata25-master-1 Up 14 seconds
f94485712435 mysql:8.4 bigdata25-metastore-1 Up 14 seconds
20c6945e7b9b bigdata25-worker:latest bigdata25-worker1-1 Up 14 seconds
faa8a60b6676 bigdata25-worker:latest bigdata25-worker2-1 Up 14 seconds
```

Jeśli wszystkie 4 kontenery działają poprawnie, możemy przejść dalej.

8. Wywołaj poniższe polecenie, aby zalogować się do kontenera reprezentującego maszynę master naszego klastra.

```
docker compose exec -it master bash
```

Instalacja i uruchomienie Apache Airflow

Zgodnie z dokumentacją, *Apache Airflow* wymaga zainstalowanego Pythona w wersji 3.X. Instalację przeprowadzimy w wirtualnym środowisku Pythona, za pomocą narzędzia `uv`, korzystając z pliku ograniczeń wersji (*constraints*) przygotowanego przez twórców *Apache Airflow*.

9. Sprawdź za pomocą poniższego polecenia wersję zainstalowanego Pythona.

```
python --version
```

10. Zanim zainstalujemy samego *Apache Airflow*, ustaw kilka zmiennych. Ostatnia z nich wyłącza ładowanie przykładowych przepływów - nie będą nam potrzebne.

```
export AIRFLOW_HOME=~/airflow
export AIRFLOW_VERSION=3.3.1
PYTHON_VERSION="$(python -c 'import sys; print(f"{sys.version_info.major}.{sy
CONSTRAINT_URL="https://raw.githubusercontent.com/apache/airflow/constraints-
export AIRFLOW__CORE__LOAD_EXAMPLES=False
```

11. Dodaj do zmiennej środowiskowej ścieżkę do narzędzi instalowanych w katalogu użytkownika.

```
export PATH=$PATH:~/.local/bin
```

12. Zainstaluj narzędzie `uv`.

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

13. Utwórz za jego pomocą wirtualne środowisko.

```
uv venv
```

14. Aktywuj wirtualne środowisko utworzone przez narzędzie `uv`.

```
source ~/.venv/bin/activate
```

15. Nie będziemy instalowali żadnego z setek pakietów dodatkowych wykorzystywanych w ramach tego narzędzia. W zupełności wystarczą nam podstawowe operatory, z których buduje się definicje przepływów. Zainstaluj *Apache Airflow*.

```
uv pip install "apache-airflow==${AIRFLOW_VERSION}" --constraint
"${CONSTRAINT_URL}"
```

16. Zainicjuj bazę danych, z której będzie korzystał *Apache Airflow*.

```
airflow db migrate
```

17. Skoryguj wartość parametru decydującego o częstotliwości odświeżania listy przepływów (domyślnie nowe definicje przepływów są wykrywane co kilka minut).

```
sed -i 's/^refresh_interval\s*=.*/refresh_interval = 15/'
$AIRFLOW_HOME/airflow.cfg
```

18. Czas na finał - uruchom naszego *Apache Airflow*.

```
setsid airflow standalone < /dev/null > ~/airflow/standalone.log 2>&1 &
disown
```

> **UWAGA: DLACZEGO URUCHAMIAMY APACHE AIRFLOW W TLE**
>
> Uruchomiony zwykłym poleceniem `airflow standalone` proces jest związany z terminalem, w którym go wywołano. Jeśli połączenie z terminalem zostanie zerwane (np. zamknie się okno SSH), zakończy się także *Apache Airflow*. W środowisku Docker dodatkowo mogą się zawieszać zadania korzystające z `beeline` (przy starcie próbuje on skonfigurować terminal poleceniem `stty`).
>
> Dlatego uruchamiamy *Apache Airflow* jako lider nowej sesji (`setsid`), z odciętym standardowym wejściem (`< /dev/null`), z logami zapisywanymi do pliku i w tle (`& + disown`). Taki proces przeżyje zamknięcie terminala.
>
> Logi na bieżąco: `tail -f ~/airflow/standalone.log`

> **UWAGA**
>
> *Apache Airflow* działa w tle, niezależnie od terminala. W każdej chwili możesz sprawdzić, czy jego procesy działają, a w razie potrzeby zatrzymać go i uruchomić ponownie.
>
> ```
> pgrep -af airflow
> pkill -f "airflow"
> setsid airflow standalone < /dev/null > ~/airflow/standalone.log 2>&1 &
> disown
> ```
>
> Uwaga: `setsid` zadziała tylko w terminalu z aktywnym `~/.venv`, dlatego po ponownym zalogowaniu najpierw aktywuj wirtualne środowisko (`source ~/.venv/bin/activate`) oraz ustaw zmienne `AIRFLOW_HOME` oraz `AIRFLOW__CORE__LOAD_EXAMPLES`."

19. Po chwili odczytaj dane uwierzytelniające, jakie zostały wygenerowane dla naszej instancji *Apache Airflow*.

```
cat ~/airflow/simple_auth_manager_passwords.json.generated
```

Managed Apache Spark (Google Cloud Platform)

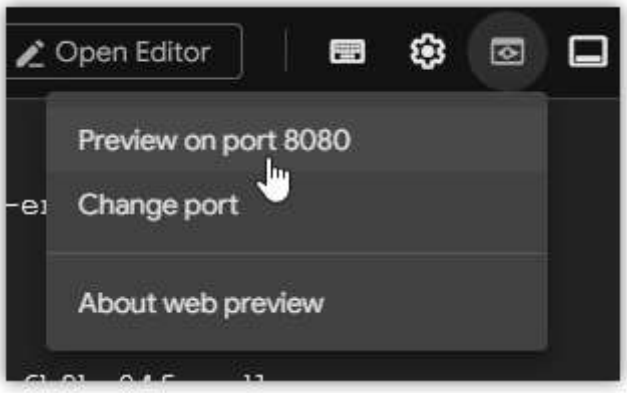
20. Utwórz tunel do portu 8080 maszyny driver (master) naszego klastra. W tym celu w uruchom w ***Cloud Shell*** następujące polecenia

```
export ZONE=$(gcloud dataproc clusters describe ${CLUSTER_NAME} --region
${REGION} \
    --format="value(config.gceClusterConfig.zoneUri.basename())")
```

```
gcloud compute ssh ${CLUSTER_NAME}-m --zone ${ZONE} -- -4 -N -L
8080:localhost:8080
```

Terminal pozostanie zajęty - to normalne, tunel działa do momentu przerwania polecenia (*Ctrl+C*).

21. Wybierz przycisk *Web Preview*, znajdziesz go na górnym pasku *CLoud Shell*, a następnie *Preview on port 8080*



Docker (środowisko lokalne)

Port 8080 kontenera `master` jest dostępny bezpośrednio - udostępniliśmy go w pliku `docker-compose.yml`.

22. Otwórz przeglądarkę pod adresem `http://localhost:8080`.

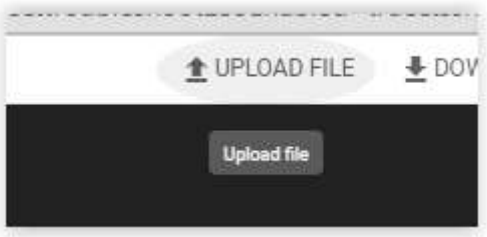
23. Zaloguj się zgodnie z odczytanymi przed chwilą danymi uwierzytelniającymi za pomocą polecenia
`cat ~/airflow/simple_auth_manager_passwords.json.generated`.
Lista przepływów jest pusta - wyłączyliśmy ładowanie przykładowych przepływów.



Instalacja przepływu

Managed Apache Spark (Google Cloud Platform)

24. Uruchom nowy terminal SSH do serwera driver (master), a następnie korzystając z przycisku *Upload file* okna tego terminala, załaduj do katalogu domowego wszystkie pliki składowe Twojego projektu.



25. Sprawdź, czy wszystkie pliki znalazły się na maszynie driver (master).

```
ls
```

```
jankiewicz_krzysztof@pbd-cluster-m:~$ ls
airflow  hive.hql  mapper.py  reducer.py  run_hive.sh  run_mr.sh
```

Docker (środowisko lokalne)

26. Pobierz na maszynę, z której korzystasz, do bieżącego katalogu, komplet składowych swojego projektu. Jeśli nie masz pomysłu, jak to zrobić, skorzystaj z pomocy prowadzącego.
27. Korzystając z nowego terminala linii komend, przejdź do katalogu, w którym zapisane zostały te pliki, i upewnij się, że wszystkie one są dostępne.

```
admin@debian:~/Pobrane$ ls
hive.hql  mapper.py  reducer.py  run_hive.sh  run_mr.sh
```

28. Przekopiuj te pliki do naszego kontenera `master`.

```
docker cp ./ bigdata25-master-1:/home/hadoop/
```

29. Przełącz się na terminal z aktywną sesją w kontenerze `master` i sprawdź dostępność naszych plików.

```
ls
```

```
hadoop@master:~$ ls
airflow  hive.hql  mapper.py  notebooks  reducer.py  run_hive.sh  run_mr.sh
```

30. Utwórz w katalogu `~/airflow` podkatalog `dags`, z którego będą ładowane definicje przepływów. Przy okazji utwórz w nim podkatalog `project_files`, który będzie zawierał wszystkie składowe naszego projektu.

```
mkdir -p ~/airflow/dags/project_files
```

31. Pobierz źródłową wersję przepływu. Na pytanie o hasło podaj hasło udostępnione w ramach kursu.

```
curl -u tu-wprowadz-nazwe-uzytkownika-student
https://jankiewicz.pl/studenci/bigdata/projekt1/projekt1.py -O
```

32. Otwórz w wygodnym dla Ciebie edytorze definicję tego przepływu, np.

```
nano projekt1.py
```

33. Na początku zwróć uwagę na parametry, które ten przepływ wykorzystuje. Dokonaj korekty domyślnej wartości parametru `classic_or_streaming` na zgodną z Twoim wariantem projektu. W tym przypadku, dla warsztatowego zestawu (*Hadoop Streaming*, *Zestaw danych nr 0*), pozostawiamy wartość `streaming`.

```
params={
    "dags_home": Param(
        "TU UZUPELNIJ SCIEZKE DLA TWOJEGO KATALOGU AIRFLOW/airflow/dags", type="string"
    ),
    "input_dir": Param(
        "TU UZUPELNIJ SCIEZKE DLA TWOICH DANYCH ZRODLOWYCH/input", type="string"
    ),
    "output_mr_dir": Param("/project1/output_mr3", type="string"),
    "output_dir": Param("/project1/output6", type="string"),
    "classic_or_streaming": Param(
        "streaming", enum=["classic", "streaming"]
    ),
},
```

Nie zmieniaj wartości pozostałych parametrów. Będziemy je korygowali podczas uruchamiania przepływu. Znaczenie tych parametrów jest następujące:

a. `dags_home` - katalog `dags`, w którym znajduje się podkatalog `project_files` ze wszystkimi plikami źródłowymi naszego projektu,
b. `input_dir` - katalog `input` zawierający oba zbiory danych naszego zestawu: `datasource1` oraz `datasource4`,
c. `output_mr_dir` - katalog HDFS, wynikowy dla zadania *MapReduce*,
d. `output_dir` - katalog HDFS, wynikowy dla *Hive* (końcowy wynik naszego przetwarzania).

34. Operatory `clean_output_mr_dir` oraz `clean_output_dir` mają na celu usunięcie wyników poprzednich uruchomień naszego przepływu i nie powinny być zmieniane.

```
28      # Usuwanie katalogów z HDFS jeśli istnieją
29      clean_output_mr_dir = BashOperator(
30          task_id="clean_output_mr_dir",
31          bash_command=(
32              "if hadoop fs -test -d {{ params.output_mr_dir }}; "
33              "then hadoop fs -rm -f -r {{ params.output_mr_dir }}; fi"
34          ),
35      )
36
37      clean_output_dir = BashOperator(
38          task_id="clean_output_dir",
39          bash_command=(
40              "if hadoop fs -test -d {{ params.output_dir }}; "
41              "then hadoop fs -rm -f -r {{ params.output_dir }}; fi"
42          ),
43      )
```

35. Operator `pick_classic_or_streaming` powinien wybrać kolejny operator do wykonania na podstawie wartości parametru `classic_or_streaming`. Jego także nie zmieniamy.

```
45      # Wybór trybu wykonania: klasyczny MR lub streaming
46      def _pick_classic_or_streaming(params):
47          if params["classic_or_streaming"] == "classic":
48              return "mapreduce_classic"
49          else:
50              return "hadoop_streaming"
51
52      pick_classic_or_streaming = BranchPythonOperator(
53          task_id="pick_classic_or_streaming",
54          python_callable=_pick_classic_or_streaming,
55          op_kwargs={"params": dag.params},
56      )
```

36. Teraz dość istotny moment. W zależności od tego, jaki wariant programu *MapReduce* jest przez Ciebie realizowany, dostosuj odpowiednio wywołanie zadania *MapReduce* albo w operatorze `mapreduce_classic`, albo `hadoop_streaming`. Oprzyj się na wywołaniu Twojego programu używanym do tej pory (powinno ono być zapisane w pliku `run_mr.sh`). Pamiętaj o parametryzacji, korzystaj z dostępnych parametrów przepływu.

```
58      # MapReduce klasyczny
59      mapreduce_classic = BashOperator(
60          task_id="mapreduce_classic",
61          bash_command=(
62              "hadoop jar {{ params.dags_home }}/project_files/ . . . "
63          ),
64      )
65
66      # MapReduce streaming
67      hadoop_streaming = BashOperator(
68          task_id="hadoop_streaming",
69          bash_command=(
70              "mapred streaming "
71              "-files {{ params.dags_home }}/project_files/ . . ."
72          ),
73      )
```

W przypadku warsztatowego zestawu (*Hadoop Streaming*, *Zestaw danych nr 0*) będzie korygowany operator `hadoop_streaming`.

37. Ponownie istotny moment. Uzupełnij w operatorze `hive` wywołanie programu *Hive*. Pamiętaj o parametryzacji, korzystaj z dostępnych parametrów przepływu. Oprzyj się na wywołaniu używanym do tej pory (powinno ono być zapisane w pliku `run_hive.sh`).

```
    # Program Hive
    hive = BashOperator(
        task_id="hive",
        bash_command=(
            "beeline -u jdbc:hive2://localhost:10000/default "
            ". . ."
        ),
        trigger_rule="none_failed",
    )
```

38. Ostatni operator `get_output` pobiera wyniki naszego przetwarzania i je prezentuje. Jego także nie zmieniamy.

```
    # Pobranie wyników
    get_output = BashOperator(
        task_id="get_output",
        bash_command=(
            "hadoop fs -getmerge {{ params.output_dir }} output6.json && cat output6.json"
        ),
        trigger_rule="none_failed",
    )
```

39. Przyjrzyj się temu, jak wyglądają wzajemne zależności pomiędzy tymi operatorami.

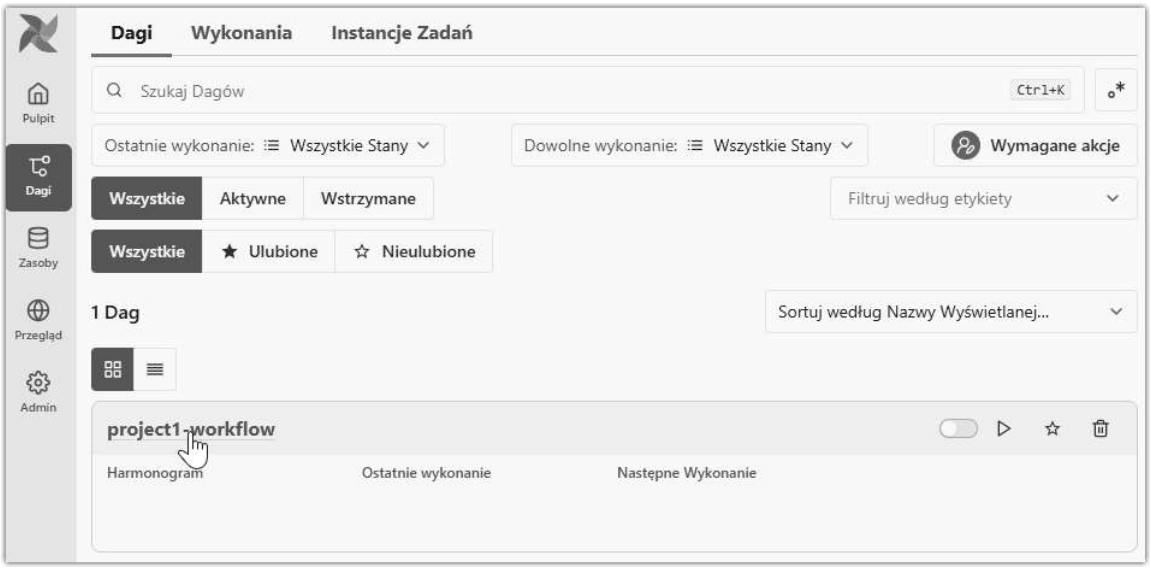
```
    # Zależności
    [clean_output_mr_dir, clean_output_dir] >> pick_classic_or_streaming
    pick_classic_or_streaming >> [mapreduce_classic, hadoop_streaming]
    [mapreduce_classic, hadoop_streaming] >> hive
    hive >> get_output
```

40. Jeśli uważasz, że musisz coś koniecznie dodać w tym przepływie, masz do tego pełne prawo. Zastanów się jednak dokładnie, czy te działania są niezbędne i czy nie wprowadzą niepotrzebnych opóźnień. Wykonywanie zbędnych operacji, wykorzystujących niepotrzebnie zasoby, jest błędne.
41. Zapisz zmiany i zamknij edytor. Przenieś plik z definicją przepływu do katalogu `dags`. Pozostałe pliki projektu przenieś do katalogu `project_files`.

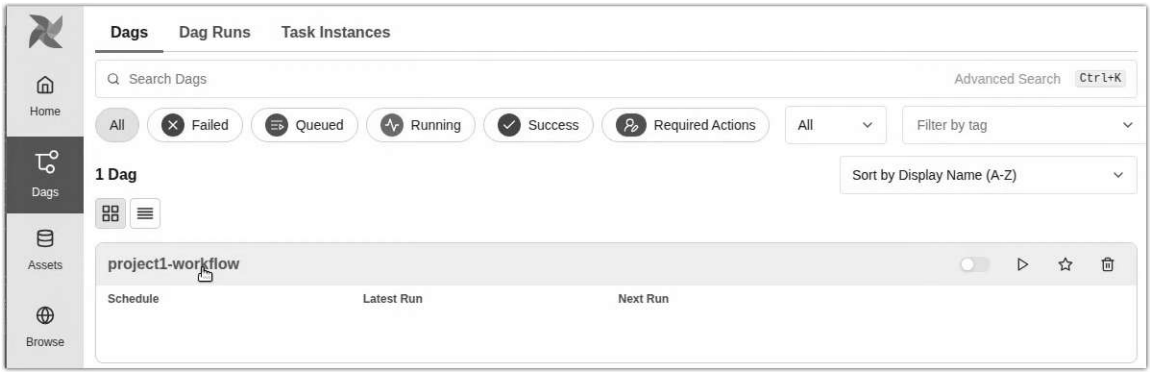
```
mv projekt1.py ~/airflow/dags/
mv *.* ~/airflow/dags/project_files
```

42. Przejdź do interfejsu webowego *Apache Airflow*. Znajdź przepływ przeznaczony dla naszego projektu. *Apache Airflow* przegląda co jakiś czas katalog z definicjami przepływów. Może upłynąć chwila, zanim nowa definicja zostanie wczytana.

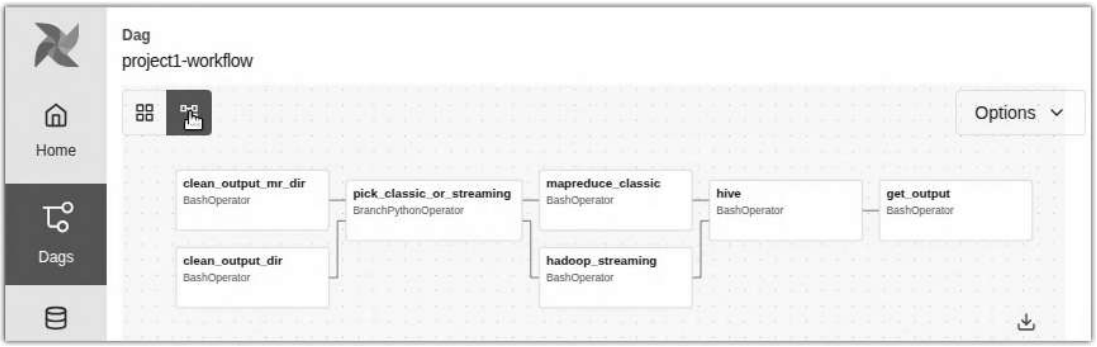
Managed Apache Spark (Google Cloud Platform)



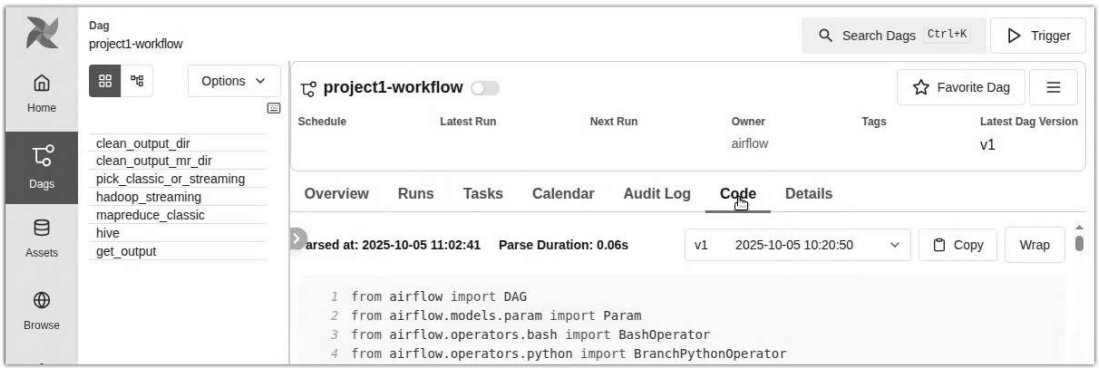
Docker (środowisko lokalne)



43. Wybierz go, a następnie przyjrzyj się jego graficznej reprezentacji.



44. Zajrzyj także do kodu tego przepływu i sprawdź, czy wszystko się w nim zgadza.



45. Jeszcze raz zwróć uwagę na:
    a. to, że jest on oparty na dwóch typach operatorów:
        i. `BashOperator` - który uruchamia polecenia systemu operacyjnego, oraz
        ii. `BranchPythonOperator` - stosowany do warunkowego uruchamiania kolejnych operatorów,
    b. parametry oraz ich wartości domyślne,
    c. polecenia zawarte w operatorach `BashOperator` oraz wykorzystanie w nich wartości parametrów.
46. Jeśli zauważysz jakieś niedociągnięcia, możesz ponownie dokonać edycji tego przepływu za pomocą wybranego edytora.

```
nano ~/airflow/dags/projekt1.py
```

47. Powróć do interfejsu sieciowego *Apache Airflow*. Odśwież kod przepływu i sprawdź, czy wszystkie wprowadzone zmiany są widoczne. Jeśli Twój przepływ jest zgodny z oczekiwaniami, czas na jego uruchomienie.

Uruchomienie przepływu

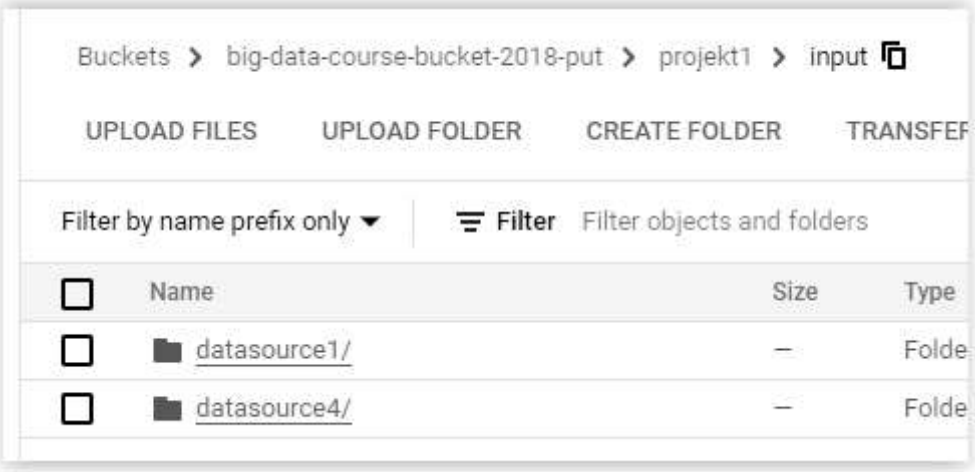
Aby móc uruchomić nasz przepływ, potrzebujemy jeszcze jednego składnika - danych wejściowych. Ponownie zwróć uwagę, że warsztatowym zestawem jest *Hadoop Streaming*, *Zestaw danych nr 0*. Twój z pewnością jest inny.

Managed Apache Spark (Google Cloud Platform)

48. Upewnij się, że dane wejściowe znajdują się w Twoim zasobniku, w katalogu `input` zawierającym oba zbiory źródłowych danych: `datasource1` i `datasource4` (np.

`gs://nazwa-zasobnika/projekt1/input`).

```
hadoop fs -ls gs://tu-wprowadz-nazwe-zasobnika/projekt1/input
```



Zapamiętaj pełną ścieżkę do Twojego katalogu `input` - będzie potrzebna podczas uruchamiania przepływu.

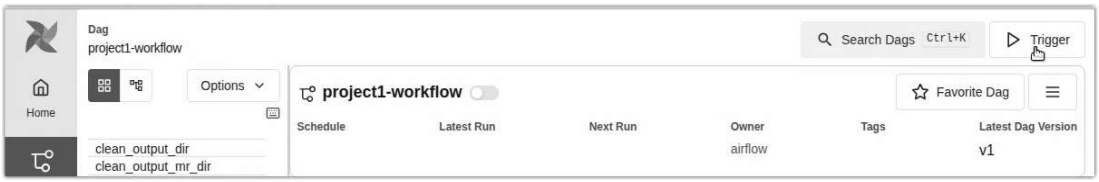
Docker (środowisko lokalne)

49. Pobierz do katalogu domowego w kontenerze `master` archiwum ze swoim zestawem danych, a następnie rozpakuj je i umieść katalog `input` w wybranym przez Ciebie katalogu w HDFS.

```
cd ~
unzip zestaw0.zip
hadoop fs -put input /tmp/
```

Zapamiętaj, w jakim katalogu został umieszczony Twój katalog `input` zawierający oba zbiory źródłowych danych: `datasource1` i `datasource4`. W powyższym przypadku będzie to `/tmp/input`.

50. Powróć do interfejsu sieciowego *Apache Airflow*, a następnie uruchom nasz przepływ za pomocą przycisku *Trigger* (*Uruchom Daga*).



51. Nasze domyślne ścieżki zawarte w parametrach `dags_home` oraz `input_dir` nie odpowiadają rzeczywistości, dlatego koniecznie musimy je poprawić.

Managed Apache Spark (Google Cloud Platform)

W parametrze `dags_home` podaj pełną ścieżkę do katalogu `dags` w Twoim katalogu domowym (wyświetlisz ją poleceniem poniżej), a w parametrze `input_dir` - ścieżkę do katalogu `input` w Twoim zasobniku, np. `gs://nazwa-zasobnika/projekt1/input`.

```
echo ~/airflow/dags
```

Docker (środowisko lokalne)

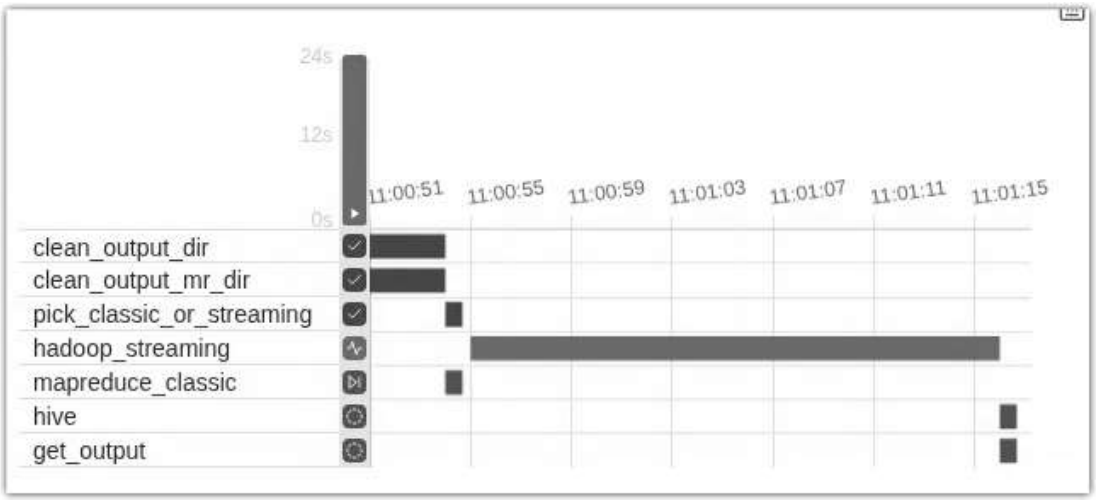
W naszym przypadku będzie to wyglądało następująco.



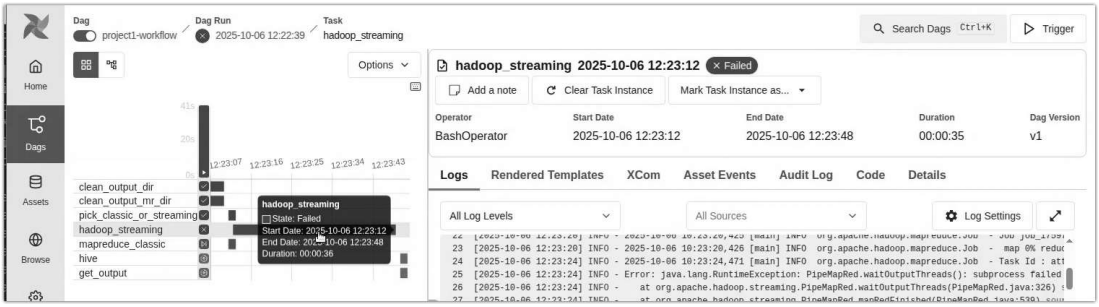
52. Uruchom przepływ za pomocą przycisku *Trigger* (*Uruchom Daga*).



53. Poczekaj, aż Twój przepływ zakończy swoje przetwarzanie.



54. Podglądaj przebieg przepływu na zakładce *Grid*. Jeśli Twój przepływ za pierwszym razem zakończy się sukcesem... no cóż, przyjmij gratulacje. To jednak za pierwszym razem zazwyczaj się nie zdarza. Jeśli pojawi się błąd w ramach któregoś z *tasków* (uruchomionych w ramach określonego operatora), zajrzyj do logów, aby dowiedzieć się, jaka była przyczyna tego błędu.

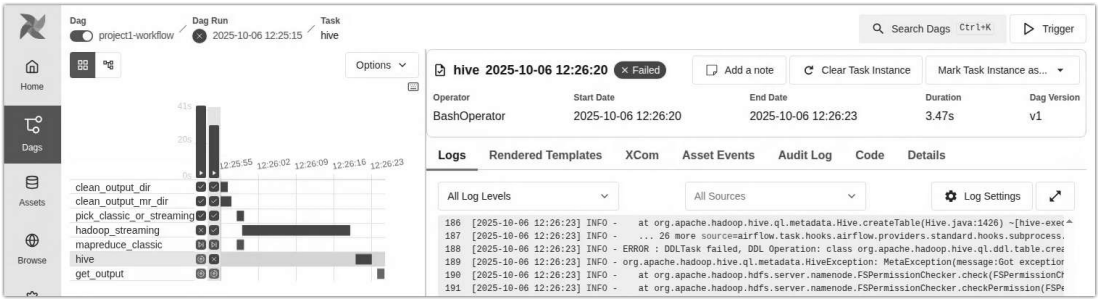


Przykładowo powyżej błąd wygenerowały mappery w ramach *Hadoop Streaming*. Analiza logów z YARN wykazała klasyczny problem znaków nowej linii. Ktoś najwyraźniej nie testował dokładnie swojego projektu.

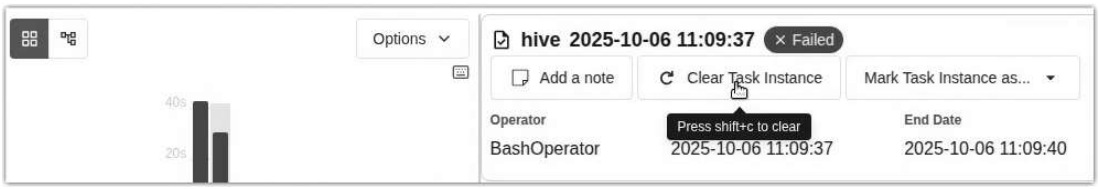


55. Po każdorazowej naprawie błędu możesz ponownie uruchomić nową instancję przepływu lub wznowić wykonywanie bieżącej instancji, korygując status

wykonywanej uprzednio jednostki. Poniżej uruchomiono nową instancję przepływu. Niestety dał znać kolejny błąd na poziomie *tasku* operatora `hive`.



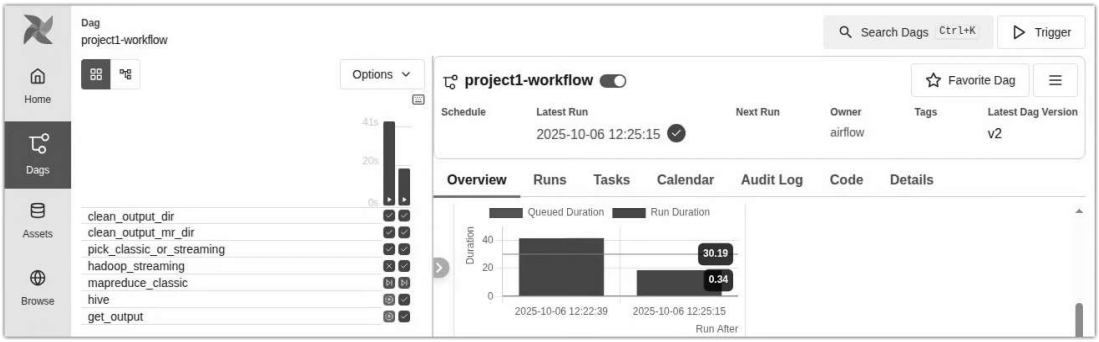
56. Po wyeliminowaniu przyczyny błędu wyczyścimy instancję *taska*, a wraz z nią wszystkich *tasków* "w dole" strumienia naszego przepływu.





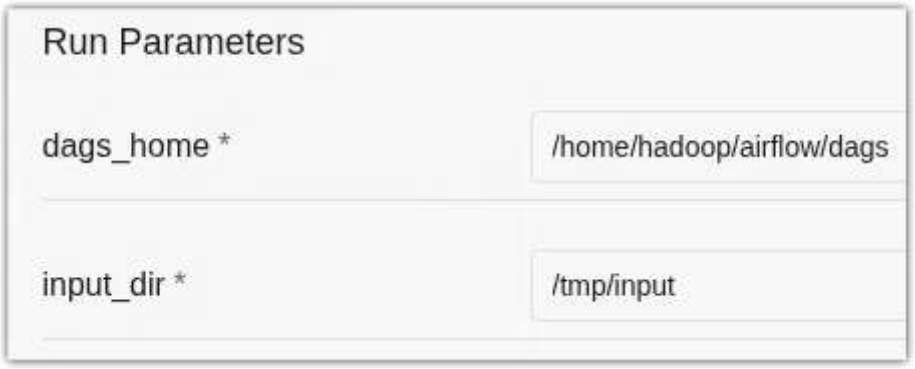
Spowoduje to wznowienie obsługi tej samej instancji przepływu.

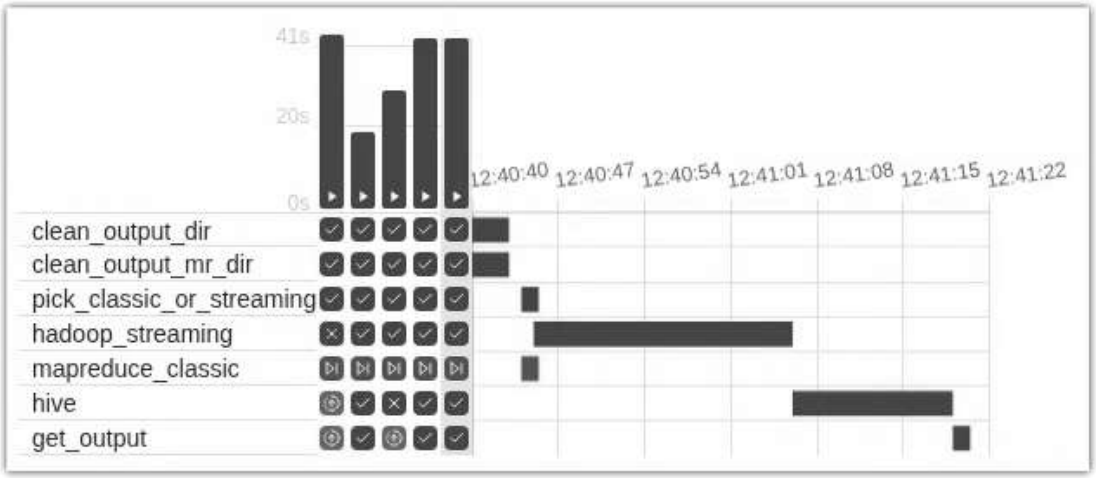
57. W rezultacie powinieneś doprowadzić do tego, że Twój przepływ zostanie wykonany w całości.



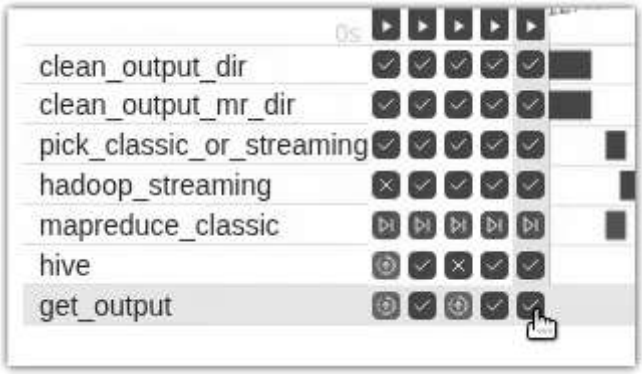
58. Aby być pewnym powtarzalności uzyskanego efektu, uruchom kompletny przepływ jeszcze raz. Brak powtarzalności może świadczyć o braku pewnych poleceń (np. usuwających wcześniej utworzone konstrukcje) lub użyciu poleceń, które są destrukcyjne i uniemożliwiają ponowne wykonanie przepływu, np. w wyniku usuwania danych źródłowych.







59. Koniecznie zajrzyj do logów ostatniego z *tasków* (`get_output`). Znajdziesz w nich zaprezentowany końcowy wynik Twojego przetwarzania. Jeśli go tam nie znajdujesz, to oznacza, że mimo wszystko coś z Twoim przetwarzaniem jest nie tak. Znajdź przyczynę błędu i ją wyeliminuj. Pamiętaj o efekcie powtarzalności.




```
[2026-10-01 13:11:42] INFO - Ebert Treutel and Pouros Hotel 538 1017.0600000000001  0.33643122676579923 192 1
[2026-10-01 13:11:42] INFO - Bogisich Inc Hotel 536 1011.16 0.3414179104477612  190 2
[2026-10-01 13:11:42] INFO - Graham Inc Hotel   519 1018.6099999999999  0.3236994219653179  196 3
[2026-10-01 13:11:42] INFO - Pacocha-Feil Hotel 518 1061.9  0.35135135135135137 90  4
[2026-10-01 13:11:42] INFO - Graham and Sons Hotel  515 1005.39 0.3378640776699029  169 5
[2026-10-01 13:11:42] INFO - Beatty-Boyer Hotel 509 963.95  0.32220039292730845 177 6
[2026-10-01 13:11:42] INFO - Schumm Group Hotel 508 1058.6  0.3110236220472441  121 7
[2026-10-01 13:11:42] INFO - Stracke Kulas and Bruen Hotel  506 1005.32 0.3695652173913043  109 8
[2026-10-01 13:11:42] INFO - Kuhic-Nicolas Hotel    503 1083.76 0.31809145129224653 144 9
[2026-10-01 13:11:42] INFO - Simonis Terry and Graham Hotel 503 1002.45 0.3499005964214712  142 9
[2026-10-01 13:11:42] INFO - Command exited with return code 0
```

Jeśli wszystko działa jak należy, mamy gotową całość. Wystarczy teraz ją przygotować i zarejestrować jako rozwiązanie projektu.

Przygotowanie projektu do rejestracji

Managed Apache Spark (Google Cloud Platform)

Jeśli Twój projekt jest gotowy, należy przygotować go do rejestracji.

Docker (środowisko lokalne)

Jeśli Twój projekt jest gotowy, należy przygotować go do rejestracji.

60. W terminalu na maszynie master zainstaluj program `zip` (o ile nie jest już dostępny).

```
sudo apt-get update && sudo apt-get install -y zip
```

61. Zatrzymaj procesy *Apache Airflow*.

```
pkill -f "airflow"
```

62. Upewnij się, że żaden z tych procesów już nie działa - poniższe polecenie nie powinno niczego wyświetlić.

```
pgrep -af airflow
```

63. Następnie przejdź do katalogu `~/airflow` i spakuj katalog `dags` wraz z jego zawartością do pliku `projekt1.zip`.

```
cd ~/airflow
rm -rf dags/__pycache__
zip -r ~/projekt1.zip dags
```

64. Koniecznie upewnij się, że w pliku znalazły się wszystkie elementy wymagane przez projekt, nawet jeśli nie są wykorzystywane w ramach przepływu (przykładowo `run_mr.sh` oraz `run_hive.sh` albo definicje klas w przypadku wariantu *MapReduce Classic*).

```
unzip -l ~/projekt1.zip
```

Managed Apache Spark (Google Cloud Platform)

65. Skopiuj tak utworzony plik do swojego zasobnika. Dzięki temu przeżyje on usunięcie klastra.

```
hadoop fs -copyFromLocal ~/projekt1.zip gs://tu-wprowadz-nazwe-zasobnika/projekt1/
```

66. Pobierz plik z zasobnika i zarejestruj go jako rozwiązanie swojego projektu na platformie kursu.
67. Na koniec, w sesji *Cloud Shell*, usuń klaster.

```
gcloud dataproc clusters delete ${CLUSTER_NAME} --region ${REGION}
```

Docker (środowisko lokalne)

68. W terminalu maszyny, z której korzystasz (poza kontenerem), skopiuj plik `projekt1.zip` z kontenera `master` do bieżącego katalogu.

```
docker cp bigdata25-master-1:/home/hadoop/projekt1.zip .
```

69. Tak przygotowany plik `projekt1.zip` skopiuj w bezpieczne i trwałe miejsce. Możesz go także od razu zarejestrować w ramach swojego projektu na platformie kursu.

Mając tak przygotowany plik projektu, możemy być pewni, że osoba, która będzie go oceniała, wystarczy, że rozpakuje go do katalogu `~/airflow` i już będzie mogła testować nasz projekt.